

Blurb

Retro Programming for the Modern(ish) Software Developer

by Mark Dalrymple. markd@borkware.com

Copyright 2025 Mark Dalrymple. All rights reserved.
Please do not train LLMs on this PDF's contents.

You've used IDEs, programmed in typesafe languages with highly featured rich standard libraries, and huge amount of complexity.

You've seen (or grew up with) the older computer systems that do way less, but look like they're a huge amount of fun. Nothing gets in the way between you and being able to exploit all the hardware, and being able to push the hardware far beyond what the original creators envisioned.

I grew up with the Apple][, Apple //e, and early Mac. I learned to program from code listings from magazines (and debugging the errors I typed in), and had a huge amount of fun, enough to make me want to program for a

living (having to choose between music and programming as I was leaving high school).

I want to share some of the fun, pain, glory, and enjoyment that comes from programming these older machines.

over-arching ideas

- * Learn BASIC *and* 6502 assembly at the same time.
- * Get introduced to something in BASIC, and then see something similar (if possible) in assembly. Or maybe related tangentially
- * aimed for experienced coders. Won't cover the theory of what a variable, loop, function, etc is. more on how

Good lord, why?

- * compare these specs of a modern laptop and a not-so-modern lappy

M4 Powerbook

- 14" x 9.8" x 0.6"
- 4.7 lb
- M4 Max processor
- 64 bits
- 4.3 Ghz clock
- 95 billion transistors
- 16 cores
- 66 registers per core
- 64gb RAM
- 1TB storage
- 3456x2234 color screen
- October 2024
- \$4,199
- fun factor: 4/10

Model 100

- 11.5" x 8" x 2"
- 3.1 lb
- Intel 80C85
- 8/16 bits
- 2.4 Mhz clock
- 6,500 transistors
- 1 core
- 11 registers per core
- 8-32K RAM
- 8-32K RAM
- 8x40 char / 240x64 px monochrome
- March 1983
- \$1,399 (4,200 today)
- fun factor: 8/10

or about 7 orders of magnitude in the number of transistors, and a massive leap.

But, programming the modern machines really isn't that much fun any more (at least for me). You've got horrible IDEs, huge toolkits that take a lot of effort to start using, knowledge of a complex programming language with lots of sharp corners

Where with BASIC, on the model 100 for example, I was able to use the cursor keys to move a little stick figure of a human being around on screen, and it only took about N lines of code:

And that was *fun*. The limited capacity of the machine's computational abilities, coupled with just how (relatively) slow they were and the rather paltry amount of storage, means that even if you wanted to, writing a large sprawling complex bloating program was kind of hard. You'd run out of resources (and patience) long before you were done unless you were extremely patient and organized.

But little hacks? Small programs to do exactly what you need? smol games that are fun? writing a two-liner to confuse and impress your friends? so much fun.

And so MarkD, why this odd trip down memory lane? These were my formative years. Going from an 8-bit apple][to a 512 mac to unix machines to the computational monsters we have in our pockets and on our desktops.

But I never really **understood** things back then. I was able to write some real software to help power my Dad's radiology and nuclear medicine departments (which was kind of scary in retrospect - a teenager writing a program

to track radioactive isotopes). But now I know stuff. I've worked on software big and small. I've swooped in on client projects for short stints, and have worked on a single piece of software for 15 years.

Now I understand stuff - like how BASIC tokenized and stored its program. What soft-switches really are, and just how joyful and fun it is to program a machine that (if you wanted to) you could learn just about **everything** about it, all the way up and down its hardware and software stack. Imagine trying to do that with modern Mac with xcode and Swift and SwiftUI. Granted, you can do a whole lot more with that, with the accompanying increase in complexity, and removal from the actual technology that does the work.

So this pile of nonsense, assuming I actually ever finish it, will take you on a journey learning the high-level stuff (yay BASIC), the low-level stuff (yay 6502 assembly language) and the stuff in-between. And sometimes might even give you some insight of how and why things work the way they do now.

Intro

There's a bunch of different ways to program these older platforms.

You can go for the original experience, by finding or purchasing a retro system, such as a fully functioning Apple][, and program on that.

You can go for the original experience with conveniences, by purchasing a retro system, and add-on hardware that lets you use an SD card as floppy or a hard drive. That way you don't need to source blank diskettes, and things generally get a lot more convenient by being able to download and use disk images.

You can also go for the basic emulator route, which is free (or very low price depending on the emulator package you're using). There are also super emulators (my term) for programs that will give you all sorts of telemetry about the machine as it runs (virtually) - examine RAM while running, take snapshots of memory, see visualizations of

anything useful to a developer.

I'm aiming for folks going the original-hardware or simple emulator route, to give the flavor of what it was like back in the day to program these beasts (especially as a hobbyist / amateur programming), along with the pain points involved. (debugger? what's that? compiling, what's that? type safety? what's that?) If you decide to write serious software for these platforms (whether to sell, or give away for bragging rights), you'll probably want more powerful tools to help you out.

I'm planning on targeting the Apple][platform (since that's what I grew up on, from sixth grade through 10th grade until we got a 512K Mac for the household), and the TRS-80 Model 100 (a recent acquisition, but it's a delightful, if slow, portable computer)

OBTW, currently the chapters are in no real order.

IDEs of March

You're probably used to high quality stable IDEs like

Visual Studio (code) or Android Studio, or move-in editors like emacs or the vi-family.

Editing code on the old machines typically was "move the cursor back, make your change, and retype the rest of the line" or "retype the entire line". When machines started getting **massive** amounts of memory (like 128K), some editing utilities were added. But before then, LIST and retying lines was the thing to do.

source code control

Source code control. What's that? It's hard to imagine, but there was a time where ubiquitous source code control wasn't the norm. Only professional shops did that, and sometimes they didn't use tools like git - they would have a cronjob that would tar everything up and put it in /tmp.

(translation - an automated process, driven by time, say every hour, would archive a shared directory of code and then put it into a temporary directory.) maybe the archive would get saved to another disk or sent to tape. Wanted to rollback a change? You'd need to go to one of the recent

archives, extract your file, and then copy it into the directory hierarchy. This may sound terrible, but I actually encountered this in a team at a Big Wallstreet Firm that made a toolkit of high-performance networking that was used to get current stock value from the exchanges in the early 90s)

For things like the programs that we write, source control is saving under different names and making copies of floppies. There were no diff tools, so you couldn't really tell what the difference between two programs were.

(research note - see if there were any source tools and diffing available)

Data Structures

You're probably used to having a rich set of data structures available. Arrays. Dictionaries / Maps. And what you don't have you can create your own through structures and objects.

In BASIC. You have three data structures:

- * numbers

- * strings
- * arrays

And that's it. You can have variables of different numeric types. You can have strings of characters (no longer than 255, though). And you can have uniform-typed arrays.

Want a structure? You'll need a small pile variables
Want an array of structures? You'll need several parallel arrays.

So let's dig in to organizing data in BASIC

In assembly, the "pointer plus offset" addressing mode was made for creating structures.

Algorithms

Algorithms

In addition to data structures, you're probably used to a rich set of algorithms. Map. Filter. Min, Max, Sorting, Searching.

With 8K of ROM for BASIC, algorithms weren't on

anyone's radar. If you want to sort something, you got to bubble sort an array (or maybe use something fancy like a shell sort.). And don't forget, recursion isn't a thing.

But there is a certain joy in knowing how the basic algorithms work at a visceral level by implementing them

Program Organization and Control Flow

Functions. Function Pointers. Methods. RPC. Recursion. Indirection. Such elegant solutions to many program organization design puzzles. Oh, and the delicious kinds of control flow. for loops, while loops, do until. tail recursion.

In BASIC-land, you have a fairly robust amount of control flow options, even if they aren't quite as pretty as what you're used to. There's GOTO (absolute branch), GOSUB (branch plus return, the closest you'll get for custom functions), FOR for doing loops with a changing numeric index, and IF for branching. Many basics don't have an ELSE

You may have heard "goto considered harmful". Put that out of your mind, because GOTO is one of your main sources of control flow when making runtime decisions

You think you've seen spaghetti code? It ain't nothing compared to the horrors you can make with GOTO.
(include example horror)

Graphics

RGB images with 8 bits per red / green / blue channel. Alpha compositing. Postscript drawing models with bezier paths, gaussian blur, and overwrought distorted glass effects. Gigantic screens with millions upon millions of pixels. GPUs that can pump out 120 fps of immersive textured environments. Those are all modern inventions.

Back in the day, we had 1920 chonky pixels that could hold one of sixteen colors, our lo-res graphics. 36,120 smaller pixels in our hi-res graphics that could hold one of eight colors (with restrictions), with wild memory layouts if you wanted to interact with the bits and bytes directly. VBL was also strangely absent on some systems (especially the apple // series) making it difficult to sync your graphics flicker-free with the scanning beam of the

CRT.

The scanning what of the what?

Hi-Res Graphics

YAY MORE PIXELS! Fewer colors.

Total bizarritude of memory layout (if didn't touch last chapter), and total weirdness of apple][colors

"when you look at the weird stuff, and learn the `_why_`, you realize how clever and elegant it was, doing things that would take years later to become common place. But now we're kind of stuck with weirdness". Which the longer you do software, the less clever you tend to become.

Audio

Different retro systems had wildly different audio. Like the Model 100 has BASIC commands for doing Music. The C64 had the SID(?) chip that could do (audio information).

The Apple // had a speaker that can ... click.

That's it. To make sounds, you make the speaker click at different rates.

File I/O

Early early days, we used audio tape to save our programs and data. I won't make you go through that difficulty. When floppy disks came out, OMG so much space for activities. Need more space, just buy another \$5 floppy disk.

140k That was the massive amount. These days it's hard to get an image resource to be below 140K. But at the time, it was a MASSIVE amount of storage. You could keep DOZENS of programs on a disk.

The Apple series had two disk operating systems - DOS 3.3 (16 256 byte sectors, 35 tracks), and ProDOS. ProDOS came later and was aimed for hard drives (starting at a massive of five megabytes), having a hierarchical file system. For folks just using floppies, it was big, consumed

a ton of memory, and 5 1/4" floppies are not really amenable to subdirectories and paths.

I'm going to be using DOS 3.3 and ignoring ProDOS. If you're running a IIgs, you'll want to look for something that talks about ProDOS.

Example program: Animals

Printing

Printing. Getting stuff on paper. You ended up printing a lot because the screen was very small when dealing with large (multi-page) programs.

PR#1, sending ascii. serial configuration. control codes. Postscript? What's that? WYSIWYG? what's that?

Basic Guts

How stuff is represented in memory after tokenization, as well as runtime - how arrays and strings are represented, as well as gosub stack frames, and for loop overhead.

What's kind of amusing in retrospect, is that the primary way these old, resource constrained, slow machines were programmed was an additional layer of indirection going through the BASIC interpreter. Thereby making them even slower!

Tokenization was one way that the basic implementation (all 10K (*verify*) of it) helped compact more code into less space. Takes the line of code, replaces keywords with single byte tokens, thereby freeing up gobs of space. "PRINT" is five bytes of characters, but is a single token. (c.f. Inside the Apple //e)

How Applesoft stores variables is a great example at packing in data. (*insert diagram, etc*) . For example, variables have two bytes for their name (or a zero byte for a one-character name). Because ASCII only needs 7 bits, that means the high bit is available.

There's four types (real, integer, string, function), and there's two bits free (high bit of the first and second character of the variable name), which gives four values. The type of the variable is encoded by bits. Like reals

have both high bits off, functions have both high bits on. strings and integers have one bit on and one bit off. (c.f. Inside the Apple //e)

Strings

Strings in BASIC are pretty much "pascal strings" - length byte. They're also the only garbage collected data structure. need more than 237 characters (which I think is the max in Applesoft), then you'll need to do more work. *(and I'll need to figure out how to do that)*

in assembly-ville, it's totally up to you. need to see what things like COUT want. But if you want C-strings and spend your precious CPU cycles scanning over it to determine length, go ahead. Or maybe two-bytes length (wasteful of a precious byte), but that will let you have a string that spans the entirety of memory. Plus maybe use bank-switching

Bank Switching

The //e family introduced a card that not only gave you EIGHTY COLUMNS of text (luxury), the majority of the cards sold had an extra 64K of memory. DOUBLING the

amount of memory. You'll never run out of memory now. and there were also other Solutions that could get you up to 128K. These days I think folks have tricked out //s with a megabyte of memory. holy hand grenades batman, that is out of this world.

It uses bank-switching (learn and describe)

You can't really take advantage of that in BASIC-land, everything is pretty much nailed down in-place, and bank-switching out chunks of that are not good for the health of your program.

in assembly land, there are some ROM routines that'll copy a block of memory from main memory to the auxiliary memory, plus you can toggle the soft switches to move things around.

(project, recreate Appleworks)

Robust Software

Even though there's not much memory or speed, it's still

nice having reliable bullet-proof software. At the time, many (most) programmers were skilled amateurs who never touched a mainframe or a VAX, and the users of a lot of the software were not programmers at all. "turn key" systems were common - put in a disk, and now you're in the environment of the application. (consistent UI was still a thing that wouldn't occur for a bunch of years)

You can write software, even in BASIC, that does error checking of input, error checking of disk I/O, as well has tricks for improving performance (often at the expense of software readability) by placing variables and subroutines at specific places in your program.

Kinds of programming

BASIC is amenable to a seat-of-your-pants software development style. Just keep having fun hacking at stuff until it works.

Some folks liked to design their code, sometimes in their head, sometimes on paper. Flowcharts were a thing.

Also, different approaches - top-down (more abstract, then figure out specifics to implement it) and bottom-up (collect a bunch of tools, then assemble). Procedural programming is similar, breaking your software into a series of ~~functions~~ subroutines that you then compose for your final code.

Object-orientation, in basic (unless you have a dialect that supports it) is pretty much right-out. There's no direct support for the indirection necessary for inheritance and polymorphism. (even being able to `ON X GOSUB VARNAME` would have gone a long way, but alas you have to hardcode line numbers)

Now, in assembly land, you can do anything. For object-orientation, you'd have a table of "function pointers", even though they're not really functions. You'd index into it, and JSR to that location. An "object" can burn 2 bytes for a pointer to that jump table (and would have other metadata like superclass). The other indexed indirect addressing mode is really useful for dealing with these kinds of things.

In graphics, it can be faster to redraw in black rather than erase the screen - be very careful with the pixels that change, and don't change pixels that don't need changing. (especially with sparse line / vector type graphics)

Performance

How different BASIC constructs can affect overall performance.

c.f. Casey Muratori's interview with jetbrains, about knowing "how does this CPU like things, and then program in a way so that it gives stuff to the CPU appropriately"

Get in to the guts of how of basic interprets the tokenized stream, and how you can arrange code (e.g. forward goto vs backward, lower line numbers with frequently hit code, etc)

would be cool getting some kind of "obvious looking but pathological case" strawbork to tear apart.